Applied Bioinformatics 2026

Week11 - Create a workflow

Introduction

In the Generate a VCF file assignment you called variants for one sample.

You are now asked to automate the analysis for the project your samples originated from to process multiple samples (say up to 10)

Course materials:

- Parallel execution
- The design file
- Creating a design file

Create a workflow

Use the BioProject connected to the sample you've already analyzed. Keep the read count small enough that ten samples can finish on your computer, but the coverage should be realistically high to make sure you can detect variants.

1. Ensure that you can calls variants for a single sample.
2. Create a design.csv based on the BioProject.
3. Add a batch target that reads design.csv and launches the workflow.
4. Merge all variant calls into a single VCF file.
5. Write a README.md that a reviewer can follow.

What to put in the README

- Summarize the bioproject: how many samples, what the project was about etc.
- Describe how the run the workflow, ensure that it can be reproduced by a reviewer.
- Visualize the results in IGV or and summarize a few findings that seem relevant.

Bioproject PRJNA1066305

NCBI BioProject PRJNA1066305, titled Whole genome sequencing of Helicobacter pylori isolates, contains three H. pylori samples, with three associated BioSamples, three SRA experiments, and three genome assemblies. The project was submitted by Northern Arizona University and focuses on whole-genome sequencing of H. pylori isolates obtained from Native American patients presenting with gastric disease; the samples were collected from gastric biopsy specimens associated with the Winslow Indian Health Care Center in Arizona. The three strains represented in the project are 408F, 412F, and 427F, and the sequencing was performed as paired-end whole-genome sequencing on the Illumina MiSeq platform. The project is medically relevant because H. pylori infection is associated with gastritis and other gastric diseases, and these genome sequences provide data for studying genetic variation among clinical isolates from this patient population.

Analysis

Run the makefile to see the help and options:

```
make
```

```
#
# Variant calling with bcftools
#
# GENOME_NAME=CHC155
```

```
# GENOME_FASTA=fasta/CHC155.fasta
# SAMPLE_NAME=412F
# SRR=SRR27606652
# N=40000
#
# BAM=bam/412F.bam
# VCF=vcf/variants.vcf.gz
# STATS=vcf/variants.stats.txt
#
# Available targets:
#   make all    - process one sample and call variants
#   make batch  - process samples from design.csv in parallel and call variants
#   make igv    - visualize the variant results in IGV
#   make clean  - remove all generated files and directories
#

Have a look at first four columns of the design file:

```bash
xan select run_accession,sample_accession,sample_alias,sample_description design.csv | xan
view
```

run_accession	sample_accession	sample_alias	sample_description
SRR27606652	SAMN39484834	412F-DNA-002	Gastric biopsy from WIHCC
SRR27606651	SAMN39484835	427F-DNA-001	Gastric biopsy from WIHCC
SRR27606653	SAMN39484833	408F-DNA-001	Gastric biopsy from WIHCC

Run the batch target and output to a log file:

```
make batch > output.log
```

We initially planned to annotate the called variants with bcftools csq, which predicts coding consequences using a reference FASTA, a GFF annotation, and a VCF. However, bcftools csq works most naturally with Ensembl-style GFF3 annotations, while the available Helicobacter pylori CHC155 annotation from NCBI was NCBI-style GFF3. We attempted to convert the annotation into a compatible format, but the conversion did not work reliably. Rather than force the annotation into a format that might introduce errors, we decided to use our own variant_effect.py script to interpret the variants directly from the reference sequence and gene annotation. This provided a simpler and more transparent way to determine whether variants were synonymous, nonsynonymous, or otherwise affected coding regions while keeping the analysis reproducible.

Run the annotator developed by Greg Cermak/ChatGPT:

```
python variant_effect.py \
    fasta/CHC155.fasta \
    gff/CHC155.gff \
    vcf/variants.vcf.gz \
    > vcf/variant_effects.tsv
```

Count the number of variants for each consequence type in variant_effects.tsv:
```

```
# Count the number of variants for each consequence type in variant_effects.tsv
awk -F '\t' '!/^#/ && $1 != "CHROM" {count[$11]++} \
END {for (c in count) print c "\t" count[c]}' \
vcf/variant_effects.tsv | sort
```

```
frameshift      159
inframe_deletion        104
inframe_insertion       68
intergenic      4805
missense        15207
start_lost      16
stop_gained     165
stop_lost       49
synonymous      29696
unknown 2
```

Save the first example of each CONSEQUENCE, including the IGV_GOTO coordinate:

```
# Save the first example of each CONSEQUENCE, including STRAND and IGV_GOTO.
grep -v '^#' vcf/variant_effects.tsv | \
awk -F '\t' '
BEGIN {
    print "CHROM\tPOS\tREF\tALT\tGENE\tSTRAND\tCONSEQUENCE\tIGV_GOTO"
}
NR==1 {next}
!seen[$11]++ {
    print $1 "\t" $2 "\t" $3 "\t" $4 "\t" $5 "\t" $6 "\t" $11 "\t" $12
}' > vcf/consequence_examples.tsv
```

View the output of consequence_examples.tsv:

```
# View the output of consequence_examples.tsv
xan view vcf/consequence_examples.tsv

# or for raw markdown
xan to markdown vcf/consequence_examples.tsv
```

CHROM	POS	REF	ALT	GENE	STRAND	CONSEQUENCE	IGV_GOTO
NZ_AP026446.1	58	C	T	-	-	intergenic	NZ_AP026446.1:58
NZ_AP026446.1	139	G	A	OW489_RS00005	+	synonymous	NZ_AP026446.1:139
NZ_AP026446.1	152	G	A	OW489_RS00005	+	missense	NZ_AP026446.1:152
NZ_AP026446.1	6958	AAATGCT	A	OW489_RS00035	+	inframe_deletion	NZ_AP026446.1:6958
NZ_AP026446.1	7074	AT	A	OW489_RS00040	+	frameshift	NZ_AP026446.1:7074
NZ_AP026446.1	8825	T	A	OW489_RS00045	+	stop_gained	NZ_AP026446.1:8825
NZ_AP026446.1	51259	T	A	OW489_RS00215	+	stop_lost	NZ_AP026446.1:51259
NZ_AP026446.1	90610	G	GTTA	OW489_RS00390	-	inframe_insertion	NZ_AP026446.1:90610
NZ_AP026446.1	123099	C	T	OW489_RS00565	-	start_lost	NZ_AP026446.1:123099

CHROM	POS	REF	ALT	GENE	STRAND	CONSEQUENCE	IGV_GOTO
NZ_AP026446.1	249797	G	A	OW489_RS01135	+	unknown	NZ_AP026446.1:249797

The output allows us to see the gene, the type of consequence and a goto for viewing in IGV!

Results

Run the makefile to get a default IGV view of the results (IGV must be running):

```
make igv
```

```
echo "new" | nc -q 1 localhost 60151
OK
echo "genome /home/user/Documents/bioinfo/cdf/fasta/CHC155.fasta" | nc -q 1 localhost
60151
OK
echo "load /home/user/Documents/bioinfo/cdf/gff/CHC155.gff" | nc -q 1 localhost 60151
OK
echo "load /home/user/Documents/bioinfo/cdf/vcf/variants.vcf.gz" | nc -q 1 localhost 60151
OK
```

Default IGV View of the Batch Analysis

The default IGV view shows the *Helicobacter pylori* CHC155 reference genome together with the combined `variants.vcf.gz`, BAM alignment tracks for strains 408F-DNA-001, 412F-DNA-002, and 427F-DNA-001, and the CHC155 GFF annotation. Across the approximately 1.7 Mb chromosome, the three BAM tracks show broadly continuous alignment coverage, while the VCF track shows variant calls distributed throughout the genome. The GFF annotation provides genomic context by showing the locations and orientations of genes. This whole-genome view provides an overview of variation across the three strains and can be used to identify regions for closer inspection.

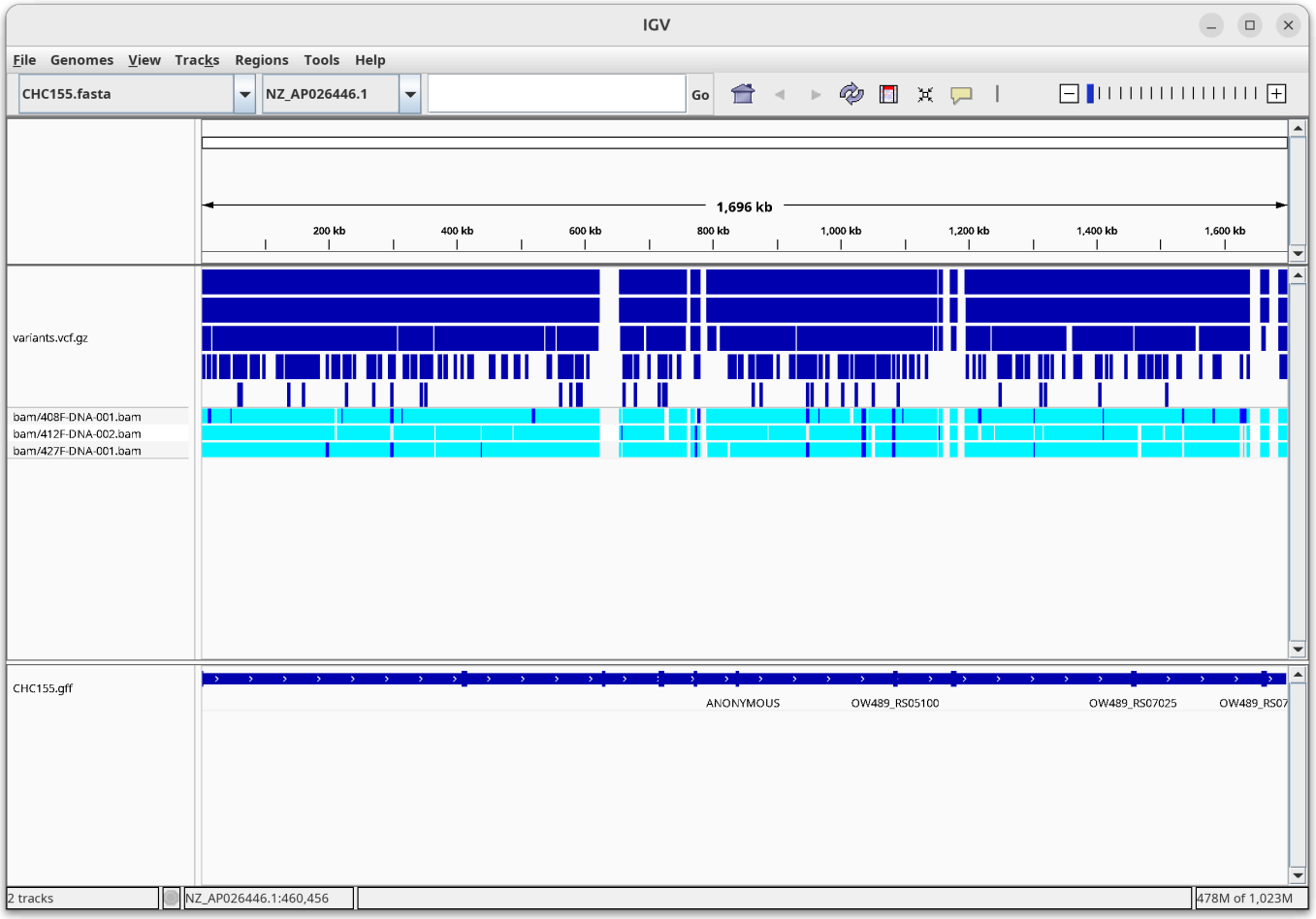


Figure: Default IGV view of the batch analysis. The CHC155 reference genome spans approximately 1.696 Mb. The upper `variants.vcf.gz` track displays variant calls across the genome, the three BAM tracks show aligned reads for 408F-DNA-001, 412F-DNA-002, and 427F-DNA-001, and the CHC155 GFF track at the bottom shows annotated genes and their orientations.

In-frame Deletion

The variant at position 6958 in `OW489_RS00035` is an in-frame deletion. The VCF reports `REF=AAATGCT` and `ALT=A`, so six bases (`AATGCT`) are deleted after the shared anchor base. Because six bases equal two complete codons, the reading frame is preserved. The predicted consequence is the loss of two amino acids from the protein, while the downstream amino-acid sequence remains in the same reading frame.

```
6 deleted bases
      ↓
AATGCT
      ↓
AAT | GCT
      ↓
2 codons
      ↓
2 amino acids removed
      ↓
reading frame preserved


Protein before:
... A - B - C - D - E - F - G ...

Protein after:
... A - B ----- E - F - G ...
```

```
                ↑
        two amino acids removed
```

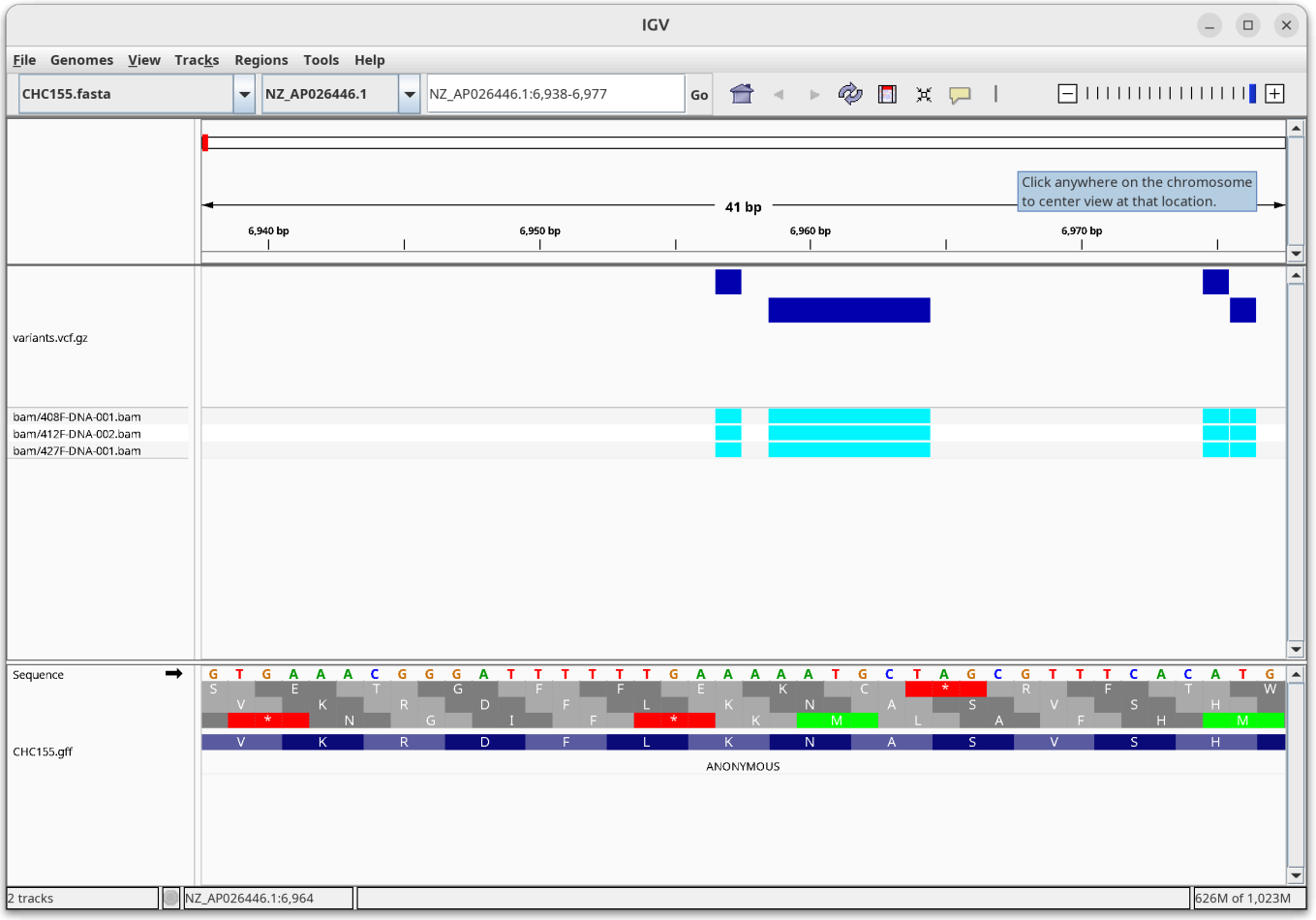


Figure: IGV view of the in-frame deletion at position 6958 in `OW489_RS00035`. The variant is represented as `AAATGCT → A`, corresponding to a six-base deletion after the shared anchor base. For sample `408F-DNA-001`, the genotype information shows a homozygous alternate call (`A/A`) with a depth of 17 and `AD=0,17`, indicating that all reads supporting the site carry the alternate allele. The six-base deletion removes two codons while preserving the downstream reading frame.

Start Lost

The variant at position 123099 in `OW489_RS00565` is a start-lost variant. The reference base changes from `C` to `T`. Because this gene is on the reverse strand, the visible reference triplet `CAT` corresponds to the coding-strand start codon `ATG`. After the variant, `CAT` becomes `TAT`, whose reverse complement is `ATA`. This changes the normal `ATG` start codon to `ATA`, so translation may no longer begin at the expected position.

```
Reference strand:
CAT
 ↓ reverse complement
ATG
 ↓
start codon

Variant:
CAT → TAT
         ↓ reverse complement
ATG → ATA
         ↓
normal start codon lost
```

```
Protein before:
START → amino acid 1 → amino acid 2 → amino acid 3 → ...

Protein after:
no normal START
      ↓
translation may fail to begin normally
or may begin at a downstream alternative start site
```

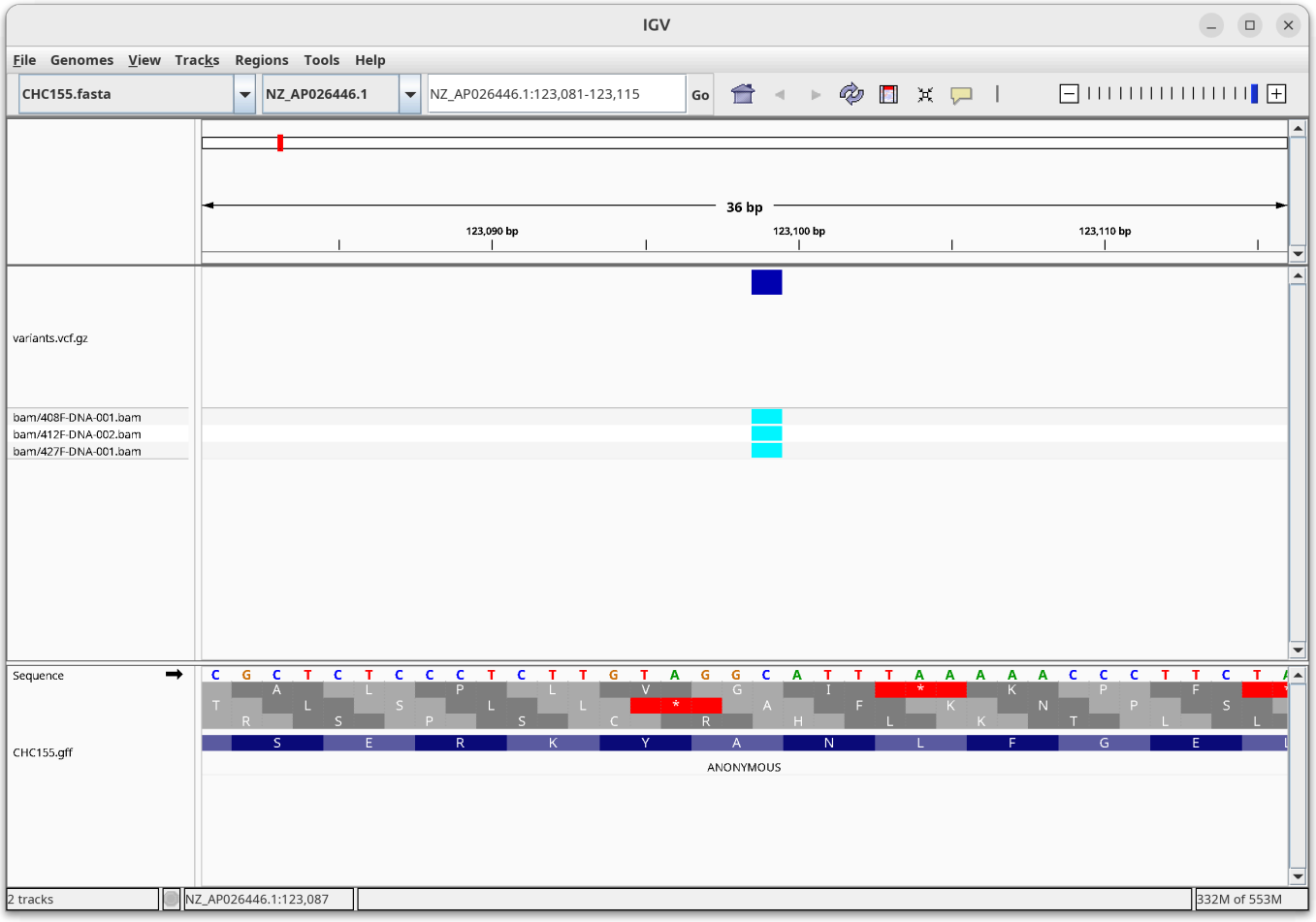


Figure: IGV view of the start-lost variant at position 123099 in `OW489_RS00565`. The reference sequence shows the triplet `CAT`, which corresponds to the reverse-strand coding sequence `ATG`, a normal start codon. The `C → T` variant changes `CAT` to `TAT`, whose reverse complement is `ATA`, eliminating the normal `ATG` start codon and producing the predicted `start_lost` consequence.

Stop Lost

The variant at position 51259 in `OW489_RS00215` is a stop-lost variant. The gene is on the `+` strand, so the codon can be read directly from the reference sequence in IGV. The reference codon is `TGA`, which is a stop codon. The `T → A` variant changes this codon to `AGA`, which encodes arginine. As a result, the normal stop signal is lost, and translation may continue into the downstream sequence until another in-frame stop codon is reached.

```
Reference:
TGA
 ↓
STOP

Variant:
TGA → AGA
       ↓
      Arg
```

```
Protein before:
... amino acid → amino acid → STOP

Protein after:
... amino acid → amino acid → Arg → continues translating
                                        ↓
                              until another stop codon is reached
```

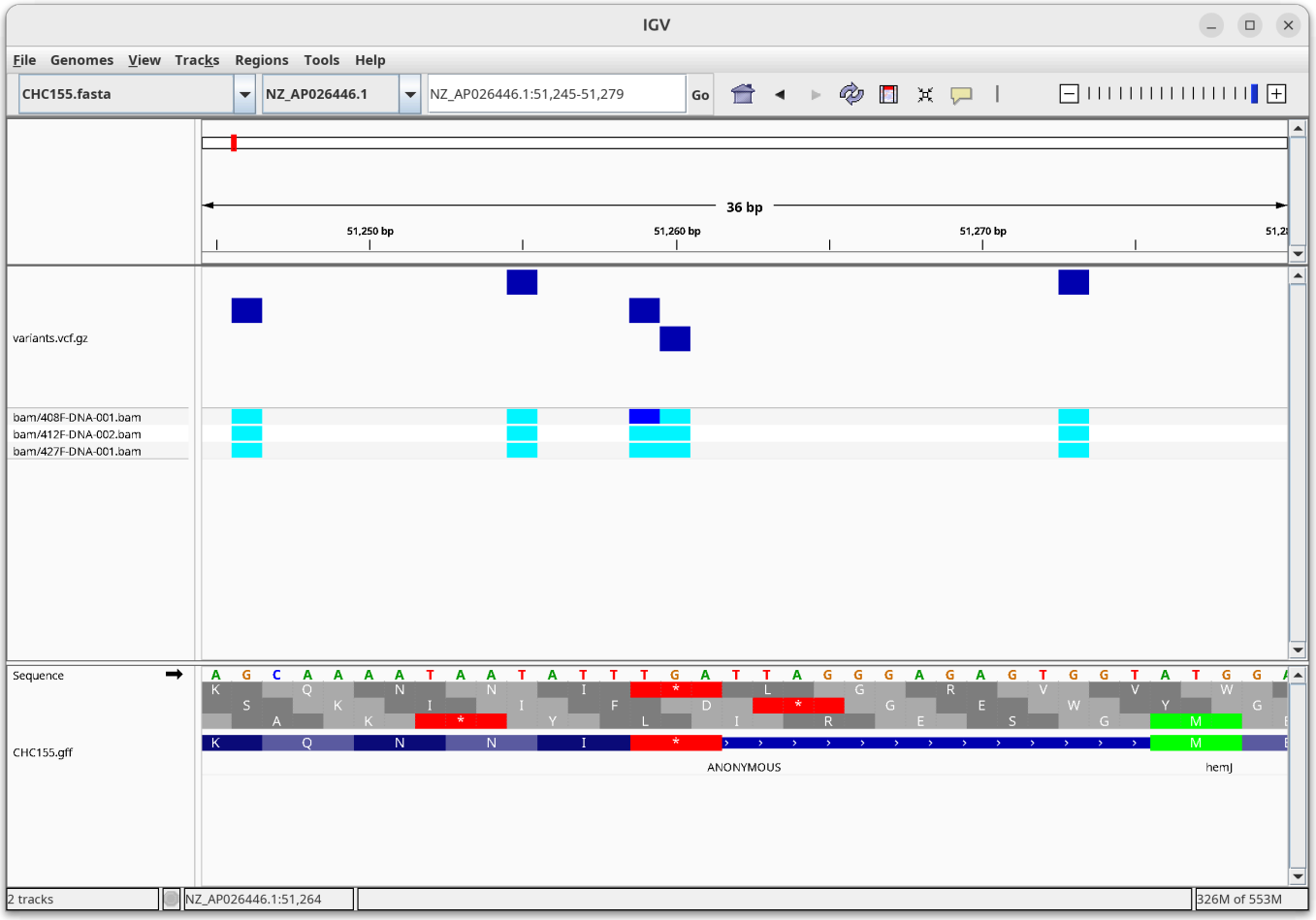


Figure: IGV view of the stop-lost variant at position 51259 in `OW489_RS00215`. The gene is on the positive strand, and the reference sequence contains the stop codon `TGA`. The `T → A` variant changes this codon to `AGA`, which encodes arginine, eliminating the normal termination signal and producing the predicted `stop_lost` consequence.

Summary

Exercise Summary

In this exercise, we built a more complete and automated variant-analysis workflow for *Helicobacter pylori* CHC155. We first created a `design.csv` file from the BioProject so multiple sequencing runs could be processed in batch, using sample metadata such as `run_accession` and `sample_alias` to launch the same workflow repeatedly without manually changing variables for each sample. The Makefile was then adapted so a single sample could be processed with `make all`, while `make batch` processed multiple samples in parallel, aligned them to the CHC155 reference genome, generated BAM files, and performed a joint variant call across all available BAMs to create one combined VCF. We then extended the workflow beyond variant detection to functional interpretation. Because the available NCBI GFF annotation was not straightforward to use with `bcftools csq`, we created a custom Python program, `variant_effect.py`, that combined the reference FASTA, GFF annotation, and VCF file to classify variants as intergenic, synonymous, missense, start-lost, stop-gained, stop-lost, frameshift, in-frame insertion, in-frame deletion, or unknown, and we later added the gene strand to the output so coding changes could be interpreted correctly. AWK was used to count consequence types and extract representative examples, which were then examined in IGV. Three examples were studied in detail: an in-frame deletion at position 6958 in `OW489_RS00035`, where six deleted bases remove two codons

while preserving the reading frame; a start-lost SNP at position 123099 in the reverse-strand gene `OW489_RS00565`, where the coding-strand start codon changes from `ATG` to `ATA`; and a stop-lost SNP at position 51259 in the positive-strand gene `OW489_RS00215`, where the normal `TGA` stop codon changes to `AGA`, encoding arginine and allowing translation to continue. Overall, the exercise connected BioProject metadata, batch processing, read alignment, joint variant calling, automated consequence prediction, strand-aware codon interpretation, and IGV visualization into a reproducible end-to-end bioinformatics workflow.